

栈 (stack): 只允许在一端 (栈顶 top) 做插入和删除的线性表

规则: 后进先出 (进栈顺序: $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_n \rightarrow a_{n-1}$ 出栈顺序: $a_{n-1} \rightarrow a_{n-2} \rightarrow \dots \rightarrow a_2 \rightarrow a_1$)

顺序栈: 用数组实现

双栈共享: 规定一个 maxSize (足够), 两端分别作为两个栈的 bottom, 往中间扩充

链式栈: top 即为 first, push, pop 操作只在链表头结点处进行 (常规操作)

*应用: ① 匹配左右括号:

思路: 设置一个栈, 从左往右看一个表达式, 每当有一个 char* == "(" 时, 就 push, 每当有一个 char* == ")" 时, 就 pop (若退栈失败, 即没有匹配的 "(", 返回失败), 最后若栈非空, 即没有足够匹配的 ")", 返回失败; 否则匹配成功。

② 后缀表达式即其计算

(中缀表达式) (后缀表达式)

(* 后缀表达式: 操作符在操作数后, eg: $A+B*(C-D)-E/F \leftrightarrow ABCD-*+EF/-$)

后缀表达式隐含了运算顺序, 故不含括号, 只有操作数和操作符

若有一个后缀表达式 char*, 计算方法: 从左往右扫描, 若为操作数则压入栈, 若为操作符则依次退栈两次 X (先) 和 Y (后), 计算 $Y <op> X$ 后将结果 R 压入栈, 最后扫描完成, 栈中还剩下的就是运算结果。

中缀表达式转化为后缀表达式算法:

操作符 ch	#	(	*, /, %	+, -	)
isp	0	1	5	3	6
icp	0	6	4	2	1

isp 叫做栈内 (in stack priority) 优先数, icp 叫做栈外 (in coming priority) 优先数。从表 3.2 中可以看到, 左括号的栈外优先数最高, 它一来立即进栈, 但当它进入栈中后, 其栈内优先数变得极低, 以便括号内的其他操作符进栈。其他操作符进入栈中后优先数都升 1, 这样可体现在中缀表达式中相同优先级的操作符自左向右计算的要求, 让位于栈顶的操作符先退栈并输出。操作符优先数相等的情况只出现在括号配对或栈底的 '#' 号与输入流最后的 '#' 号配对时。前者将连续退出位于栈顶的操作符, 直到遇到 '(' 为止。然后将 '(' 退栈以对应括号, 后者将结束算法。

扫描中缀表达式, 将它转换为后缀表达式的算法描述如下:

- ① 操作符栈初始化, 将结束符 # 进栈。然后读入中缀表达式字符流的首字符 ch。
- ② 重复执行以下步骤, 直到 ch = '#', 同时栈顶的操作符也是 '#', 停止循环。
 - a 若 ch 是操作数直接输出, 读入下一个字符 ch。
 - a 若 ch 是操作符, 判断 ch 的优先级 icp 和当前位于栈顶的操作符 op 的优先级 isp:
 - 若 $icp(ch) > isp(op)$, 令 ch 进栈, 读入下一个字符 ch。
 - 若 $icp(ch) < isp(op)$, 退栈并输出。
 - 若 $icp(ch) == isp(op)$, 退栈但不输出, 若退出的是 '(' 号读入下一字符 ch。
- ③ 算法结束, 输出序列即为所需的后缀表达式。

栈与递归 (常用递归的情况:)

1. 定义是递归的:
eg: 阶乘、数列、斐波那契
2. 数据结构是递归的:
eg: 链表 (f -> link 为一个链表, 递归到 NULL 为止)
3. 解法是递归的:
eg: Hanoi tower

递归改为非递归 (因为递归的 T 一般很高)

直接改 (单向递归) 用栈 (尾递归) -> while/for 循环

队列: 只允许在一端插入 (队尾 rear), 另一端删除 (队头 front) 的线性表

规则: 先进先出 (进队顺序 $a_1, a_2, \dots, a_n$)

顺序队列: 用数组实现的队列 (添加 front, rear 指针)

(可见出队后, 数组前面的空间就不能用了)

循环顺序队列 (逻辑上循环): 可以充分利用 front 前空的存储空间

出队 (队头指针进 1): $front = (front + 1) \% MaxSize$;
入队 (队尾指针进 1): $rear = (rear + 1) \% MaxSize$;

初始化 $front = rear = 0$; 空队列: $front = rear$; 队满: $front = (rear + 1) \% MaxSize$

链式队列

入队即从链表尾插入元素, 退队即从链表头删除元素

优点: 不限制空间大小故不会溢出; 特别适合数据元素变动大的情形

*应用: (1) 打印杨辉三角 (可扩展到所有广度优先搜索问题)

思路: 先将第 1 行 "1 1" 入队, 每行间插入 1 个 "0" 以示分行, 然后接下来将下一行的值 (利用上一行信息) 入队, 再打印上一行, 后再出队。重复上述流程即可。

队列比 = 维数组的优势: 空间复杂度从 $O(N^2) \rightarrow O(n)$

优先级队列: 每次取出的是具有最高优先权的元素 (相同优先权的元素按 FIFO 规则)

实现: 与普通队列相比, 每次入队都要放在合适的位置上 (从队尾往前找, 比优先权大小)

双端队列: 可以在队列的两端进行插入和删除。

实现: 与普通队列相比, 入队函数分为 EnQueueHead 和 EnQueueTail, 出队同理。

考点: 判断输出序列的合法性

若数据元素输入序列为 1, 2, 3, 4, 则下列哪些输出序列是合法的? 哪些是非合法的?

可以是栈、队列或者双端队列

1234 1243 1324 1342 1423 1432
2134 2143 2314 2341 2413 2431
3124 3142 3214 3241 3412 3421
4123 4132 4213 4231 4312 4321